Technical Specification: UNIhelp - Timetable Management Module

Version: 1.0
Date: October 26, 2023
Project: UNIhelp
Module: Timetable Management

1. Overview

This document outlines the technical requirements for the Timetable Management module of the UNIhelp web application. The primary goal of this module is to provide a dynamic and interactive timetable for university students, lecturers, and the management crew. The system will be built using the PERN stack (PostgreSQL, Express.js, React, Node.js).

2. User Roles & Permissions

There are three main user roles (actors) in this module, each with a distinct set of permissions.

Role	Permissions
Management Crew (Admin)	- C.R.U.D. (Create, Read, Update, Delete) on the entire timetable structure (time slots). - Assign subjects, lecturers, and locations to time slots. - Modify lecture times, dates, and locations. - C.R.U.D. for locations, including creating new locations and setting/updating their specific seat counts.
Lecturer	- Read their assigned timetable. - Update their assigned time slots to add a lecture_topic. - Update their assigned time slots to add/edit a notice. - Read the list of students who have booked a seat for their lecture. - Update the attendance status of booked seats (e.g., mark as 'attended' or 'absent'). - Download an attendance report for a completed lecture.
Student	- Read the full student timetable. - Create a seat booking for a specific time slot before the lecture begins. - Delete their own booking before the lecture begins. - Receive real-time notifications when a lecturer adds a notice to a relevant time slot.

3. Database Schema (PostgreSQL)

The following tables are proposed for the PostgreSQL database.

Table: users

- Stores user information for all roles.
- id (SERIAL, PRIMARY KEY)

- **full_name (VARCHAR)**
- **email (VARCHAR, UNIQUE, NOT NULL)**
- **password_hash (VARCHAR, NOT NULL)**
- **role (VARCHAR, NOT NULL, CHECK (role IN ('student', 'lecturer', 'admin')))**
- **created_at (TIMESTAMP, DEFAULT NOW())**

Table: subjects

- **Stores information about university subjects/courses.**
- **id (SERIAL, PRIMARY KEY)**
- **subject_name (VARCHAR, NOT NULL)**
- **subject_code (VARCHAR, UNIQUE)**

Table: locations

- **Stores information about lecture halls/rooms. Managed by Admins.**
- **id (SERIAL, PRIMARY KEY)**
- **room_name (VARCHAR, NOT NULL)**
- **seat_count (INTEGER, NOT NULL) -- Set by Admin**

Table: timeslots

- **The core table for the timetable. Each row represents a single lecture session.**
- **id (SERIAL, PRIMARY KEY)**
- **subject_id (INTEGER, FOREIGN KEY references subjects.id)**
- **lecturer_id (INTEGER, FOREIGN KEY references users.id)**
- **location_id (INTEGER, FOREIGN KEY references locations.id)**
- **day_of_week (INTEGER, NOT NULL) -- e.g., 1 for Monday, 7 for Sunday**
- **start_time (TIME, NOT NULL)**
- **end_time (TIME, NOT NULL)**
- **lecture_topic (TEXT, NULLABLE) -- Added by Lecturer**
- **notice (TEXT, NULLABLE) -- Added by Lecturer**

Table: bookings

- **Links students to a specific timeslot they have booked.**
- **id (SERIAL, PRIMARY KEY)**

- **student_id (INTEGER, FOREIGN KEY references users.id)**
- **timeslot_id (INTEGER, FOREIGN KEY references timeslots.id)**
- **seat_number (INTEGER, NOT NULL)**
- **attendance_status (VARCHAR, DEFAULT 'booked', CHECK (attendance_status IN ('booked', 'attended', 'absent'))) -- 'absent' can be used for cheaters/no-shows**
- **created_at (TIMESTAMP, DEFAULT NOW())**
- **UNIQUE (student_id, timeslot_id) -- A student can only book one seat per lecture**
- **UNIQUE (timeslot_id, seat_number) -- A seat can only be booked by one student per lecture**

Table: notifications

- **Stores notifications for students.**
- **id (SERIAL, PRIMARY KEY)**
- **user_id (INTEGER, FOREIGN KEY references users.id) -- The student receiving the notification**
- **timeslot_id (INTEGER, FOREIGN KEY references timeslots.id)**
- **message (TEXT, NOT NULL)**
- **is_read (BOOLEAN, DEFAULT FALSE)**
- **created_at (TIMESTAMP, DEFAULT NOW())**

---

4. API Endpoints (Node.js / Express.js)

All endpoints should be protected by authentication and role-based authorization middleware.

4.1. Authentication (/api/auth)

- **POST /login: Authenticate user, return JWT.**
- **GET /me: Get the current logged-in user's info from JWT.**

4.2. Management Crew (Admin) Endpoints (/api/admin)

- **Time Slot Management**
  - **POST /timeslots: Create a new time slot.**
  - **PUT /timeslots/:id: Update an existing time slot (for changing schedule/location).**
  - **DELETE /timeslots/:id: Delete a time slot.**
- **Location Management**
  - **POST /locations: Create a new location.**

        - **Body: { "room_name": "Hall A-101", "seat_count": 150 }**
    - **GET /locations: Get a list of all locations.**
    - **PUT /locations/:id: Update a location's name or seat count.**
        - **Body: { "seat_count": 160 }**
    - **DELETE /locations/:id: Delete a location.**

4.3. Lecturer Endpoints (/api/lecturer)

- **GET /mytimetable: Get the timetable for the logged-in lecturer.**
- **PUT /timeslots/:id/details: Add/update a lecture topic.**
    - **Body: { "lecture_topic": "..." }**
- **PUT /timeslots/:id/notice: Add/update a notice. This action must trigger notification creation on the backend.**
    - **Body: { "notice": "..." }**
- **GET /timeslots/:id/bookings: Get all student bookings for a specific lecture slot.**
- **PUT /bookings/:bookingId/attendance: Update the attendance status of a booking.**
    - **Body: { "status": "attended" } or { "status": "absent" }**
- **GET /timeslots/:id/report: Download an attendance report (e.g., CSV format).**

4.4. Student Endpoints (/api/student)

- **POST /bookings: Book a seat for a time slot.**
    - **Body: { "timeslot_id": 12, "seat_number": 25 }**
    - **Crucial Backend Validation:**
        1. **Verify the lecture (timeslot_id) has not already started.**
        2. **Retrieve the location_id from the timeslot.**
        3. **Retrieve the seat_count from the locations table.**
        4. **Validate that 1 <= seat_number <= seat_count. If not, return a 400 Bad Request error.**
        5. **Verify the seat is not already booked for that timeslot.**
        6. **If all checks pass, create the booking.**
- **DELETE /bookings/:id: Cancel a booking (must be owned by user and before lecture starts).**
- **GET /notifications: Get all notifications for the logged-in student.**

- **PUT /notifications/:id/read: Mark a notification as read.**

4.5. Common Endpoints (/api/timetable)

- **GET /: Get the full weekly timetable for display, customized based on user role.**

---

5. Frontend (React)

5.1. Suggested Component Structure

- **Admin Dashboard:**
  - **TimetableManagement.js: A view for Admins to C.R.U.D. time slots.**
  - **LocationManagement.js: A new view for Admins to C.R.U.D. locations and their seat counts.**
- **Shared Components:**
  - **TimetableView.js: The main grid displaying the weekly timetable.**
  - **TimeSlotCard.js: A component for a single lecture. Its action (on click) depends on the user's role.**
  - **NotificationBell.js: A component in the navbar for real-time notifications.**
- **Student Components:**
  - **BookingModal.js: A modal showing the seating layout for a location, allowing students to select an available seat.**
- **Lecturer Components:**
  - **AttendanceView.js: A view for lecturers to see a visual seating chart or list of students to mark attendance.**
  - **LecturerEditModal.js: A modal for lecturers to add topics and notices.**

5.2. Key Logic & Flow

1. **Seat Booking (Student Flow):**
   - **Student clicks a time slot in the TimetableView.**
   - **The BookingModal opens. An API call fetches the location's seat_count and the list of already booked seats for that slot.**
   - **The modal dynamically renders a grid of seats up to the seat_count, visually disabling already booked seats.**
   - **Student clicks an available seat and confirms. A POST request is sent to /api/student/bookings.**

- **The booking functionality must be disabled if the lecture's start_time has passed. This rule must be enforced on both the frontend (disabling the button) and backend (API validation).**

2. **Adding Notices & Real-time Notifications (Lecturer/Student Flow):**
    - **Lecturer adds a notice to a time slot.**
    - **The backend API saves the notice and creates a notification for every student enrolled in that subject/course.**
    - **Recommended: Use WebSockets (e.g., Socket.IO). The backend emits a new_notification event to the relevant student clients. The frontend listens for this event and updates the NotificationBell in real-time.**
3. **Attendance Marking (Lecturer Flow):**
    - **Lecturer opens the AttendanceView for a lecture.**
    - **The view fetches all bookings and displays them, perhaps as a list or a colored seating chart (e.g., Blue=Booked).**
    - **Lecturer clicks a student/seat to mark them as 'attended' (turns Green) or 'absent'/'cheated' (turns Red). Each click sends a PUT request to /api/lecturer/bookings/:bookingId/attendance.**
    - **A "Download Report" button triggers a GET request to /api/lecturer/timeslots/:id/report, which returns a CSV file.**

6. Getting Started

1. **Clone the project repository.**
2. **Set up the PostgreSQL database using the schema defined in Section 3.**
3. **Create a .env file in the server directory for database credentials and JWT secret.**
4. **Install dependencies: npm install in both the server and client directories.**
5. **Run the backend: npm run dev (in /server).**
6. **Run the frontend: npm start (in /client).**